

Applied Cloud Computing

Assignment 4 : Part 2: Docker Images (updated Oct. 2025)

The main objective of this homework assignment is for you to create Docker images via using Docker(<https://www.docker.com/>)

Pre-requirements: You need to have all docker related environment set up based on prior assignment.

Wherever you see the “**Answer:**”, it means you need to fill in some data that being asked. You will be graded based on the information you fill in.

Step 1

Let's create your own Docker images!

First make sure your Docker Desktop is running (blue whale shows up).

- In your local host OS's terminal/cmd window, create a subdirectory (ex: /CloudComputing/docker/basic-build) then cd(change directory) into that directory.
- Create a file called '**Dockerfile**' through your favorite editor with following two lines of content:

```
FROM alpine  
CMD ["echo", "hello world!"]
```

- Alpine is a minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size.
- Issue > **docker build --tag echo-hello-app .** ' in the directory with Dockerfile
 - It may require you to install some developer tools.
 - This command will build a Docker image based on Alpine Linux then execute one command 'echo "hello world!" '
 - You will see few lines of output during the build. The last line will be something like this "naming to docker.io/library/echo-hello-app" , " unpacking to docker.io/library/echo-hello-app:latest"
 - Check how many images are in local repository.
 - Issue > **docker image ls**
 - You should see there is one more image with that **echo-hello-app** being created.
- **Answer:** (Screenshot of your cmd output 'docker image ls')

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3323 basic-build % docker image ls
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
quay.io/centos/centos  stream9      0299bfb0830a     2 hours ago      251MB
echo-hello-app       latest       10dee6d2ba44     12 days ago      13.3MB
ubuntu               latest       66460d557b25     2 weeks ago      139MB
hello-world          latest       6dc565aa6309     2 months ago     16.9kB
quay.io/centos/centos  stream8      20da069d4f81     17 months ago    351MB
centos                8            a27fd8080b51     4 years ago      380MB
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3323 basic-build %
```

- Issue > ***docker run --name test echo-hello-app***

- What's the output of your Docker container?

▪ **Answer:**

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3323 basic-build % docker run --name test echo-hello-app
hello world!
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3323 basic-build %
```

- Congratulation! You have built your first docker image and run a container based on that image.

Step 2

Let's make this container do more things.

- Create a new file called '**script.sh**' in the same working directory with the following two lines:

```
#!/bin/sh
echo hello world, from a script file!
```

- Save script.sh and change file mode into executable by executing '**chmod +x script.sh**' in cmd prompt.
- Edit existing Dockerfile and modify the content to: (don't copy and paste, type in each line-by-line)

```
FROM alpine
COPY script.sh /script.sh
CMD ["/script.sh"]
```

- The copy command will copy the script.sh file we just edited into the container's (that we will going to create) root directory and name it script.sh (make sure the command being executed with the CMD is within a pair of " ")
- Now repeat the docker image build process via '**docker build --tag script-hello-app .**' cmd.

- Remove the previous 'test' container from the container list first by '**docker container rm test**' then run the newly build docker image via '**docker run --name test script-hello-app**'

- What's the output of this newly built container?

Answer:

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3323 basic-build % docker run --rm --name test script-hello-app
hello world, from a script file!
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3323 basic-build %
```

- What's the differences compare to previous step?

- Answer:

In Step 1, the container simply echoed a hard-coded message defined directly in the Dockerfile (CMD ["echo", "hello world!"]).

In Step 2, the container executed an external script (script.sh) that was copied into the image. This demonstrates how to package and run your own files inside Docker, making the image more flexible and powerful.

Step 3

Let's make this container do even more things...

- Modify the script.sh file again and add the 'top' command at last line:

```
#!/bin/sh
echo hello world, from a script file!
top
```

- Now repeat the docker image build process via '**docker build -tag script-hello-top-app .**' cmd. (watch out for the tag name change)

- Remove the previous 'test' container from the container list first by '**docker container rm test**' then run the newly build docker image via '**docker run --name test script-hello-top-app**'

- What's the output of this newly built container?

- Answer:

```
Mem: 673912K used, 7351788K free, 556K shrd, 13544K buff, 281828K cached
CPU:  0% usr  0% sys  0% nic 99% idle  0% io  0% irq  0% sirq
Load average: 0.16 0.14 0.09 1/244 7
```

PID	PPID	USER	STAT	VSZ	%VSZ	CPU	%CPU	COMMAND
1	0	root	S	1696	0%	1	0%	{script.sh} /bin/sh /script.sh
7	1	root	R	1692	0%	3	0%	top

- What's the differences compare to previous step?
 - Answer: Step 2 ran the script and exited immediately after printing the message. In Step 3, the script prints the message **and then keeps the container running** by starting top, so you get a live process view and the container stays attached/interactive instead of exiting.
- From the 'top' command, what's is "PID" and what is the PID for /script.sh?
 - Answer: 1
- Can you use Ctrl-C to stop the /script.sh command and why? How do you stop the 'top' command and shut down the container properly? (you need to do some googling yourself)
 - Answer: You can use **Ctrl-C** while inside the container to stop the top process, because Ctrl-C sends a SIGINT signal to the foreground process, and once top exits the script ends, which also stops the container since /script.sh is PID 1. The cleaner way to stop top is to simply press **q** to quit, which ends top and causes the script to finish. To shut down the container properly from the host side, you can run `docker stop test`, which sends a SIGTERM to PID 1 and stops the container gracefully.

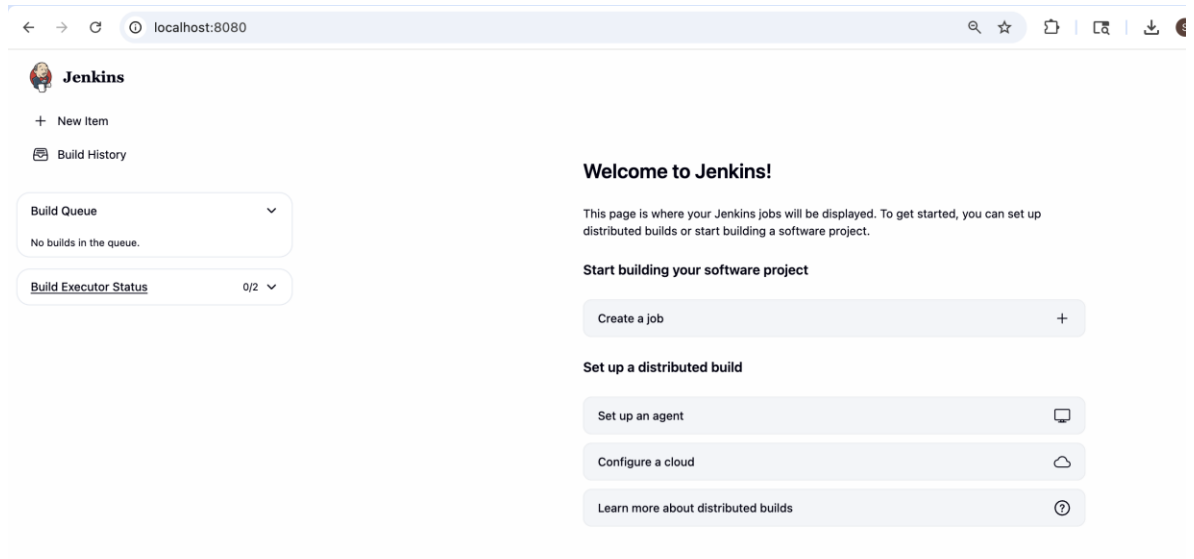
Step 4

Let's install Jenkins image then bring up a Jenkins container

- First pull the Jenkins image down to local repository/
 - > ***docker pull jenkins/jenkins:its***
- After the download completed, check local Docker images
 - > ***docker image ls***
 - you should see the Jenkins something like this:

```
jenkins/jenkins    its  82a2134e1742  13 hours ago  847MB
```

- Before we bring up Jenkins, what is Jenkins?
 - Answer: Jenkins is an open-source automation server used for CI/CD (continuous integration and continuous delivery). It automatically builds, tests, and deploys your code whenever you push changes, so teams catch bugs early and ship faster. It runs on the JVM, is highly extensible with hundreds of plugins (GitHub, Docker, Kubernetes, Slack, etc.), and lets you define pipelines as code in a Jenkinsfile to standardize and version your build/deploy steps.
- Now bring up Jenkins
 - > ***docker run --name my_jenkins -p 8080:8080 -p 50000:50000 jenkins/jenkins***
 - You need to copy a 'password' from the middle of the build log (something like "aa70df0c7e1b4f0896ea10c42eba7ce5"), later you need this key to 'unlock' Jenkins from the web UI.
 - The run process will bring up Jenkins server in the background, you need to keep this process running.
- After the Jenkins container is up, use web browser issue > <http://localhost:8080> then you should be able to connect to Jenkins web UI.
- Find the password from the docker run cmd in last step, copy and paste to the web UI in order to continue the Jenkins set up.
- Follow the web UI screen, browse then select few plug-ins to install (no need to do full install because we are not actually going to use Jenkins in next steps), use your own name to create admin user account, then start Jenkins
- When all installations are done, capture the screenshot and paste it here (to prove you had successfully brought up the Jenkins Docker container).
 - Answer: (screenshot)



- You can stop the container by issue following command from another terminal window:

`docker container stop my_jenkins`

Step 5

Let's containerize a Python App..

- We want to deploy the following Python code, contained in **index.py**. The application is a simple, "Hello World" app that uses **Flask**. [Flask](#) is a micro web framework written in Python.
- First create a directory, for example, named 'ContainerizePython'. In that directory, create **index.py** file with the following content:

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def hello():
    return "Hello World!"
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int("5000"), debug=True)
```

- Next, create another file called **Dockerfile** in the same directory with the following code.

```
FROM python:alpine3.7
COPY . /app
WORKDIR /app
```

```
RUN pip install -r requirements.txt
EXPOSE 5000
CMD python ./index.py
```

- The last file to create is **requirements.txt** (in the same directory) with following:

```
flask
```

- Note that FROM directive is pointing to python:alpine3.7. This is telling Docker what base image to use for the container, and implicitly selecting the Python version to use, which in this case is 3.7. Docker Hub has base images for almost all supported versions of Python
- Also note is the RUN directive that is calling PyPi (pip) and pointing to the requirements.txt file. This file contains a list of the dependencies that the application needs to run. Because Flask is a dependency, it is included as such in the requirements.txt with a simple reference.
- To build the image, run Docker build from a command line or terminal that is in the same root directory of the application.

docker build --tag my-python-app .

- This will “tag” the image my-python-app and build it. After it is built, you can run the image as a container.

docker run --name python-app -p 5000:5000 my-python-app

- Now you can type in the URL in a browser tab to see the Python code execution result in a web page:

http://127.0.0.1:5000/

- Capture the screenshot and paste it here (to prove you had successfully brought up the Hello World Python code from a container).
- Answer: (screenshot)



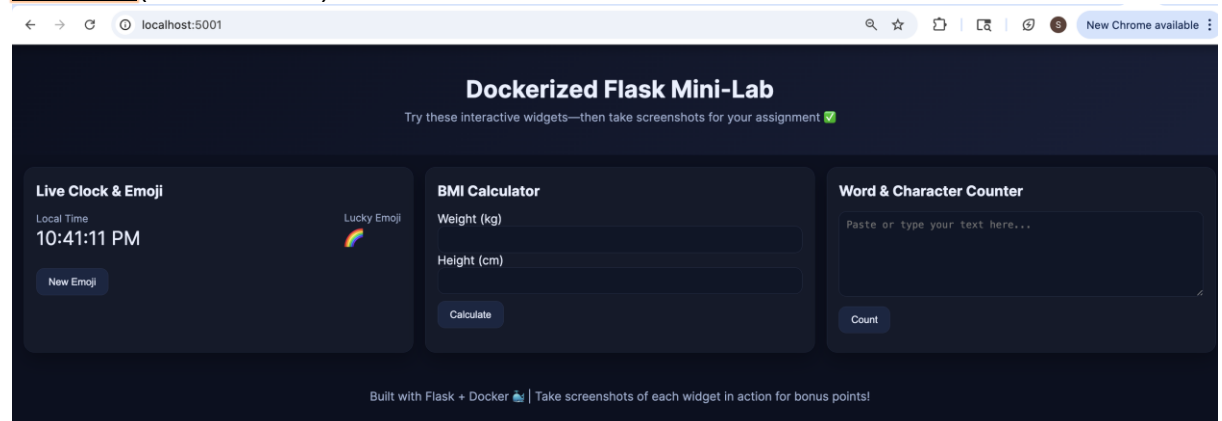
Hello World!

- You can stop the container by issue following command from another terminal window:

`docker container stop python-app`

- After completing the steps above, apply the knowledge gained from previous assignments related to Flask to enhance the `index.py` file. Create a new `index.html` following Flask's directory structure (`./templates/index.html`). Design a unique and engaging webpage—go beyond a simple “Hello World!” Use your creativity (and online resources) to develop **an interactive Web App**. Take multiple screenshots of your new webpages and paste them into the 'Answer' section. I will consider give extra bonus point for great idea.

- Answer: (screenshots)



Step 6

Docker Compose – a better way to build Docker Images

The main objective of this homework assignment is for you to learn how to use **docker compose** command to define and run multi-container Docker applications.

Step 6.1, What is 'docker compose' ?

Compose is a tool for defining and running multi-container Docker applications. With Compose, you use a YAML file(**YAML** (a [recursive acronym](#) for "YAML Ain't Markup Language") is a [human-readable data-serialization language](#). It is commonly used for [configuration files](#) and in applications where data is being stored or transmitted.) to configure your application's services. Then, with a single command, you create and start all the services from your configuration.

Using Compose is basically a three-step process:

1. Define your app's environment with a *Dockerfile* so it can be reproduced anywhere.
2. Define the services that make up your app in ***docker-compose.yml*** so they can be run together in an isolated environment.
3. Run **docker compose up** and Compose starts and runs your entire app.

Step 6.2, docker compose and WordPress

Note: WordPress is a free and open-source **content management system** written in PHP and paired with a MySQL or MariaDB database.

You can use Docker Compose to easily run WordPress in an isolated environment built with Docker containers. Below will demonstrate how to use Compose to set up and run WordPress(with 2 containers).

1. Create an empty project directory.

You can name the directory something easy for you to remember(like 'my_wordpress'). This directory is the context for your application image. The directory should only contain resources to build that image.

This project directory contains a *docker-compose.yml* file which is complete in itself for a good starter wordpress project.

Tip: You can use either a .yml or .yaml extension for this file. They both work.

2. Change into your project directory.

For example, if you named your directory *my_wordpress*:

```
cd my_wordpress/
```

3. Create a *docker-compose.yml* file (based on following example) that starts your WordPress blog and a separate MySQL instance with a volume mount for data persistence:

Take a look at *docker-compose.yml*

```
services:
  db:
    image: mysql:8.4
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: somewordpress
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
  wordpress:
    depends_on: [db]
    image: wordpress:latest
    ports: ["8000:80"]
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_NAME: wordpress
volumes:
  db_data: {}
```

The above .yml file describes a system/Microservice system which consist of two modules/containers. One is Wordpress server, the other is a MySQL database. Via using the **docker compose** command, you can build then bring up both containers together in one command.

Step 6.3, Build the project

Now, run

docker compose up -d from your project directory.

This runs “*docker compose up*” in detached mode, pulls the needed Docker images, and starts the WordPress and database containers, as shown in the example below. It will take some time to finish the build.

```
(base) Tzu-Chins-MacBook-Pro:my_wordpress atan$ docker compose up -d
WARN[0000] /Users/atan/Documents/SCU/CloudComputing/2025Winter_Cloud/Project/Part2/Proj2-docker/basic-build/my_wordpress/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 35/35
  ✓ wordpress Pulled                                40.1s
  ✓ db Pulled                                         33.8s
[+] Running 4/4
  ✓ Network my_wordpress_default                    C... 0.2s
  ✓ Volume "my_wordpress_db_data"                   Created 0.0s
  ✓ Container my_wordpress-db-1                     St... 1.5s
  ✓ Container my_wordpress-wordpress-1              Started 0.8s
(base) Tzu-Chins-MacBook-Pro:my_wordpress atan$
```

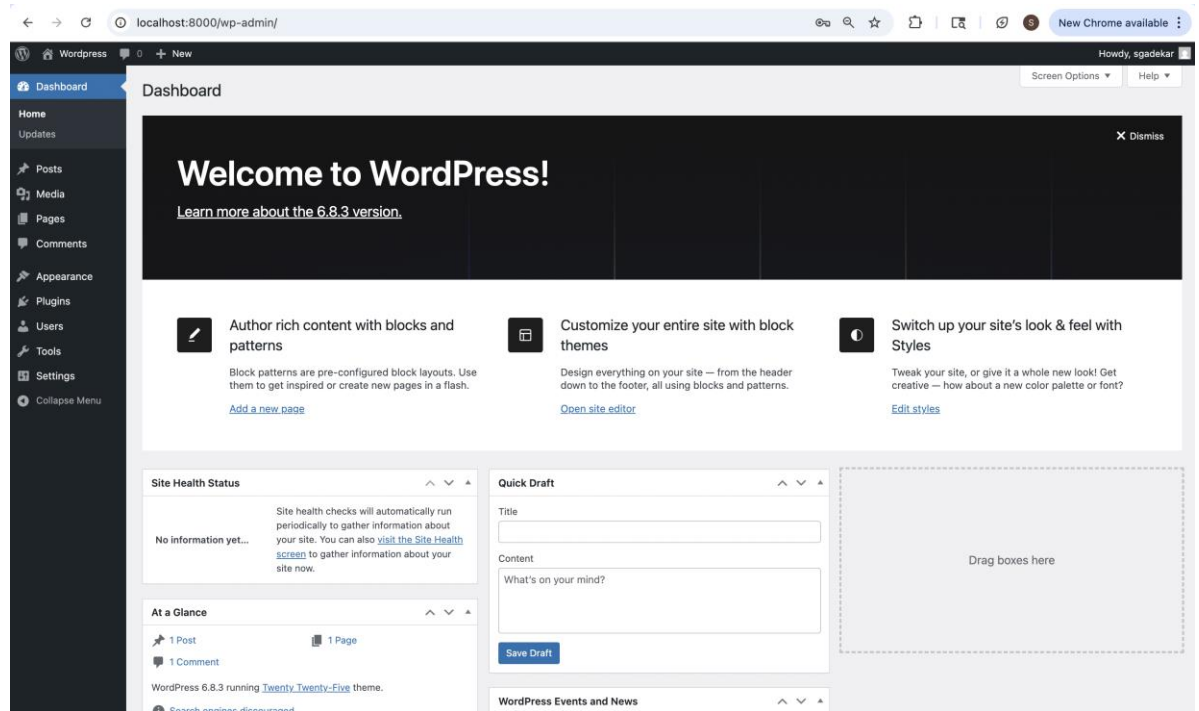
Step 6.4, Bring up WordPress in a web browser

At this point, WordPress should be running on port 8000 of your Docker Host, and you can complete the “famous five-minute installation” as a WordPress administrator.

Use a web browser, open ***http://localhost:8000*** . You should see the WordPress initial set up screen. Follow these few steps to finish the initial set up, then you will see a full WordPress content management system is up and running in your local containers, with backend MySQL also running.

Capture the WordPress initial Dashboard and paste below:

- Answer: (screenshot)



Use the WordPress feature, create a site then publish your first post with a graphic inserted as the homepage.

- Answer: (screenshot)



Wordpress

Cloud Computing

Written by [sgadekar](#) in [Uncategorized](#)

Initial Post

Step 6.5, Shutdown and cleanup

The command **`docker compose down`** removes the containers and default network, but preserves your WordPress database.

The command **`docker compose down --volumes`** removes the containers, default network, and the WordPress database.

Take a screenshot with you docker compose down (or doccker compose down – volumes) cmd.

- Answer: (screenshot)

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3323 my_wordpress % docker compose down
[+] Running 3/3
 ✓ Container my_wordpress-wordpress-1 Removed
 ✓ Container my_wordpress-db-1         Removed
 ✓ Network my_wordpress_default        Removed
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3323 my_wordpress %
```

1.3s
1.2s
0.2s

This is the end of this assignment

Extra Note 1: Automated testing environments

An important part of any Continuous Deployment or Continuous Integration process is the automated test suite. Automated end-to-end testing requires an environment in which to run tests. Compose provides a convenient way to create and destroy isolated testing environments for your test suite. By defining the full environment in a [Compose file](#), you can create and destroy these environments in just a few commands, for example:

```
$ docker compose up
```

```
$ ./run_tests (this is the test suites that you can create to perform actual testing. Here is a reference how to automatically test the WordPress website: https://codeable.io/how-to-properly-test-wordpress/ )
```

```
$ docker compose down
```

It will require lots of effort to create a working test suite. The truth is that it takes some time to create a testing script, but once you have one, it will run and test your website across a wide range of devices to make sure your core website features still work properly. A testing script has several advantages over manual testing:

- Eliminates human error - a script doesn't get bored or stressed, therefore it will catch things you'd normally miss as a human if you were to look at the same website 4 or 5 times on different browsers.
- Saves you time - long running scripts can be left alone, while you are focusing on more important items.
- Saves you more time - scripts can be executed in parallel: you can test on 4 browsers and 2 devices at the same time and get all the results at once.

Extra Note 2:

- **How to uninstall Docker:**
 - Docker Uninstall by cmd line:
 - `$ <DockerforMacPath> --uninstall`
 - `$ /Applications/Docker.app/Contents/MacOS/Docker --uninstall`
- **How to go to Docker Store:**
 - Docker menu -> More of Docker -> Docker Store
- **Why is deploying software to a docker image (if that's the right term) easier than simply deploying to a consistent production environment?**

Deploying a consistent production environment is easier said than done. Even if you use tools like Chef and Puppet, there are always OS updates and other things that change between hosts and environments.

Docker gives you the ability to snapshot the OS into a shared image, and makes it easy to deploy on other Docker hosts. Locally, dev, qa, prod, etc.: all the same image. Sure you can do this with other tools, but not nearly as easily or fast.

This is great for testing; let's say you have thousands of tests that need to connect to a database, and each test needs a pristine copy of the database and will make changes to the data. The classic approach to this is to reset the database after every test either with custom code or with tools like Flyway - this can be very time-consuming and means that tests must be run serially. However, with Docker you could create an image of your database and run up one instance per test, and then run all the tests in parallel since you know they will all be running against the same snapshot of the database. Since the tests are running in parallel and in Docker containers they could run all on the same box at the same time and should finish much faster. Try doing that with a full VM.

From comments...

Interesting! I suppose I'm still confused by the notion of "snapshot[ting] the OS". How does one do that without, well, making an image of the OS?

Well, let's see if I can explain. You start with a base image, and then make your changes, and commit those changes using docker, and it creates an image. This image contains only the differences from the base. When you want to run your image, you also need the base, and it layers your image on top of the base using a layered file system: as mentioned above, Docker uses AUFS. AUFS merges the different layers together and you get what you want; you just need to run it. You can keep adding more and more images (layers) and it will continue to only save the diffs. Since Docker typically builds on top of ready-made images from a registry, you rarely have to "snapshot" the whole OS yourself.

<https://training.docker.com/>

self-paced training: <https://training.play-with-docker.com/>

CI/CD Integrating CI/CD with Docker Enterprise Edition

<https://www.youtube.com/watch?v=gHIRpZJ768U>